
Big Data Analysis

Avance 1

Proyecto Olist: e-commerce brasileño
como caso de estudio para el mercado peruano

Integrantes:

Rios Chiuca, Jaime Arturo
Huaman Llanos, Felix Moises
Merino Huaman, Manuel
Montes Quispe, Adrian Guido

Contenido del avance:

KPIs del proyecto · Modelo conceptual · Modelo lógico · Modelo físico

Lima, 23 de abril de 2026

Introducción

El presente avance responde a la primera entrega del proyecto de consultoría analítica sobre el dataset Olist Brazilian E-Commerce. El objetivo es definir la base de trabajo sobre la cual se construirán los análisis de los cinco problemas de negocio planteados: rentabilidad por categoría, retención de clientes, control de tiempos de entrega, desempeño de vendedores e identificación de causas de insatisfacción.

Toda la arquitectura de datos que se presenta a continuación —KPIs, modelos conceptual, lógico y físico— está diseñada con un criterio único: habilitar respuestas accionables a esos cinco problemas de negocio, no simplemente describir la estructura de los datos originales.

1. KPIs del proyecto

Los indicadores clave de desempeño (KPIs) traducen las preguntas de la gerencia en métricas cuantificables. En lugar de presentarlos como una lista general del negocio, los organizamos por el problema al que responden, de forma que cada uno pueda rastrearse hasta una decisión concreta que la dirección debe tomar.

1.1 KPIs agrupados por problema de negocio

Problema 1 — Rentabilidad por categoría

KPI	Fórmula	Unidad
Ingreso total	$\sum(\text{precio} + \text{flete})$	R\$
Ticket promedio	Ingreso total / número de pedidos	R\$ / pedido
Concentración de ingreso	% ingreso del top 5 de categorías sobre el total	%
Flete sobre precio	$(\text{flete}/\text{precio}) \times 100$	%

Problema 2 — Retención de clientes

KPI	Fórmula	Unidad
Tasa de recompra	$\text{Clientes con } \geq 2 \text{ pedidos} / \text{total de clientes} \times 100$	%
Clientes únicos	Número de <code>customer_unique_id</code> distintos	Unidades
Valor de vida del cliente (CLV aproximado)	Ingreso total / clientes únicos	R\$ / cliente
Distribución RFM	Proporción de clientes por segmento (VIP, Frecuente, En riesgo, Dormido, Perdido)	%

Problema 3 — Control de tiempos de entrega

KPI	Fórmula	Unidad
Porcentaje de entregas a tiempo	$\text{Pedidos con dias_retraso} \leq 0 / \text{total de pedidos} \times 100$	%
Tiempo promedio de entrega	Promedio de dias_hasta_entrega	Días
Días de retraso promedio	Promedio de dias_retraso sobre pedidos retrasados	Días
Pedidos con retraso crítico	% de pedidos con dias_retraso > 7	%

Problema 4 — Desempeño de vendedores

KPI	Fórmula	Unidad
Rating promedio por vendedor	Promedio de puntuacion_review por id_vendedor	1 a 5
Pedidos con problemas por vendedor	% de pedidos del vendedor con puntuación ≤ 2	%
Ingreso por vendedor	$\sum \text{valor_total}$ agrupado por vendedor	R\$
Vendedores críticos	% de vendedores con rating < 3.5 y puntualidad < 80 %	%

Problema 5 — Causas de insatisfacción

KPI	Fórmula	Unidad
Porcentaje de reseñas positivas	$\text{Reseñas con puntuación 4-5} / \text{total de reseñas} \times 100$	%
Porcentaje de reseñas negativas	$\text{Reseñas con puntuación 1-2} / \text{total de reseñas} \times 100$	%
NPS estimado	% promotores (5) – % detractores (1-2)	Puntos
Evolución de satisfacción	Promedio mensual de puntuación, serie de tiempo	1 a 5

Los 21 indicadores definidos cubren las tres dimensiones clásicas del negocio —rentabilidad, cliente y operación— y a la vez responden punto por punto a las preguntas centrales del enunciado del proyecto. Todos se calculan directamente sobre las tablas del modelo que se presenta en las siguientes secciones, sin requerir transformaciones adicionales.

2. Modelo conceptual

El modelo conceptual representa las entidades del negocio y las relaciones entre ellas, sin comprometerse todavía con detalles de implementación. En el caso de Olist, el sistema gira en torno a siete entidades: Cliente, Pedido, Producto, Vendedor, Pago, Reseña y Ubicación.

2.1 Entidades y relaciones

Un cliente puede realizar múltiples pedidos (relación 1 : N), y cada pedido contiene uno o varios ítems; cada ítem asocia a un producto con el vendedor que lo despacha, lo cual genera la relación $N : M$ entre pedidos y productos. Cada pedido tiene al menos una forma de pago

asociada, y opcionalmente puede recibir una reseña por parte del cliente —por eso la relación entre Pedido y Reseña es $1 : 0..1$, no todos los pedidos generan reseña. Finalmente, tanto Cliente como Vendedor están vinculados a una Ubicación geográfica, lo cual es esencial para el análisis de rutas logísticas del Problema 3.

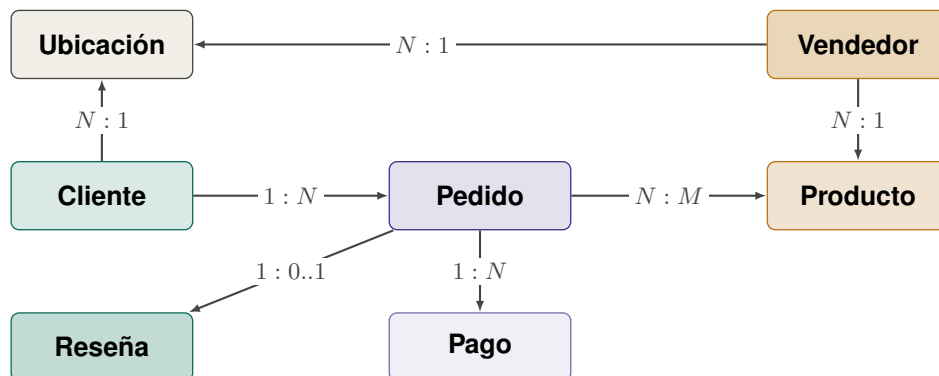


Figura 1: Modelo conceptual del dominio Olist.

2.2 Decisiones clave del modelo conceptual

- La entidad **Ubicación** se incluye explícitamente (no como atributo de Cliente o Vendedor) porque interviene en dos problemas de negocio distintos: análisis regional del cliente (Problema 2) y construcción de rutas logísticas origen→destino (Problema 3).
- La relación Pedido-Reseña es $1 : 0..1$ y no $1 : 1$: solo una parte de los pedidos genera reseña, y ese subconjunto es el que alimenta los KPIs de satisfacción del Problema 5.
- La relación Producto-Vendedor es $N : 1$ dentro de cada ítem del pedido —un producto se despacha desde un único vendedor en una transacción específica—, pero el mismo producto puede ser ofrecido por varios vendedores distintos a través del tiempo.

3. Modelo lógico

El modelo lógico traduce las entidades conceptuales a una arquitectura orientada al análisis multidimensional. Se optó por un **modelo en estrella**, donde una tabla central de hechos `FACT_VENTAS` se conecta a seis dimensiones que permiten filtrar, agrupar y pivotar la información según distintas perspectivas de negocio.

3.1 Granularidad de la tabla de hechos

El grano de `FACT_VENTAS` es el **ítem vendido**, no el pedido. Esta decisión es importante: un pedido puede contener productos de múltiples categorías y vendedores distintos, por lo que si el grano fuera el pedido se perdería la capacidad de analizar rentabilidad por categoría (Problema 1) y desempeño por vendedor (Problema 4). Con el grano a nivel de ítem, cada línea de la tabla representa un producto comprado por un cliente, despachado por un vendedor, en una fecha específica.

3.2 Diagrama del modelo estrella

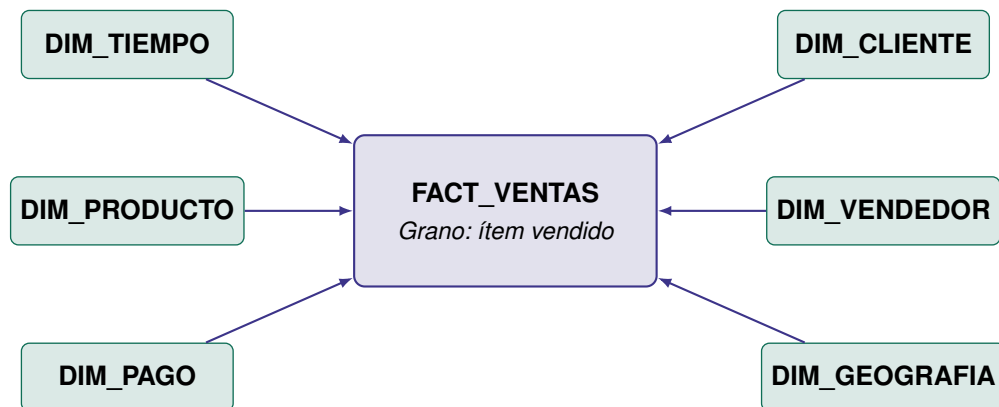


Figura 2: Modelo en estrella: tabla de hechos y seis dimensiones.

3.3 Dimensiones y sus atributos

Tabla	Contenido y propósito
FACT_VENTAS	Una fila por ítem vendido. Contiene las métricas cuantitativas del negocio: precio, flete, valor total, días hasta entrega, días de retraso, flag de entrega a tiempo, flete sobre precio, puntuación de reseña, nivel de satisfacción y flag de comentario escrito.
DIM_TIEMPO	Descomposición de la fecha en año, mes, trimestre, día de la semana y flag de fin de semana. Permite análisis temporales y estacionales (Problemas 1 y 3).
DIM_CLIENTE	Identificador del cliente y su <code>customer_unique_id</code> (necesario para calcular recompra). Enlaza a DIM_GEOGRAFIA para el análisis regional.
DIM_PRODUCTO	Categoría del producto traducida al español, rango de precio (bajo/medio/alto), peso y dimensiones. Base del Problema 1.
DIM_VENDEDOR	Identificador del vendedor y su ubicación geográfica. Base del Problema 4.
DIM_PAGO	Tipo de pago y número de cuotas. Permite cruzar comportamiento financiero con rentabilidad (Problema 1).
DIM_GEOGRAFIA	Dimensión compartida (role-playing): se conecta dos veces a la tabla de hechos, una para la ubicación del cliente y otra para la ubicación del vendedor. Esto permite construir el análisis de rutas logísticas origen→destino del Problema 3.

3.4 Variables derivadas calculadas en el ETL

Además de los campos originales de los nueve archivos CSV del dataset, el proceso de ETL genera las siguientes variables derivadas, que se materializan directamente como columnas de FACT_VENTAS:

- `valor_total` = `precio` + `flete`

- $\text{dias_hasta_entrega} = \text{fecha_entrega_real} - \text{fecha_compra}$
- $\text{dias_retraso} = \text{fecha_entrega_real} - \text{fecha_entrega_estimada}$
- $\text{entrego_a_tiempo} = 1$ si $\text{dias_retraso} \leq 0$, 0 en caso contrario
- $\text{flete_sobre_precio} = (\text{flete} / \text{precio}) \times 100$
- $\text{satisfaccion} = \text{"Buena" (4-5), "Regular" (3), "Mala" (1-2)}$
- $\text{escribio_comentario} = 1$ si el campo `review_comment_message` es no nulo, 0 en caso contrario

Materializar estas variables en la tabla de hechos (en lugar de calcularlas al vuelo en cada consulta) simplifica el trabajo del dashboard y garantiza consistencia entre vistas.

4. Modelo físico

El modelo físico lleva el diseño lógico a una implementación concreta sobre SQL Server. Se definen los tipos de datos, las claves primarias y foráneas, y los índices que optimizan las consultas más frecuentes del dashboard.

4.1 Diagrama del modelo físico

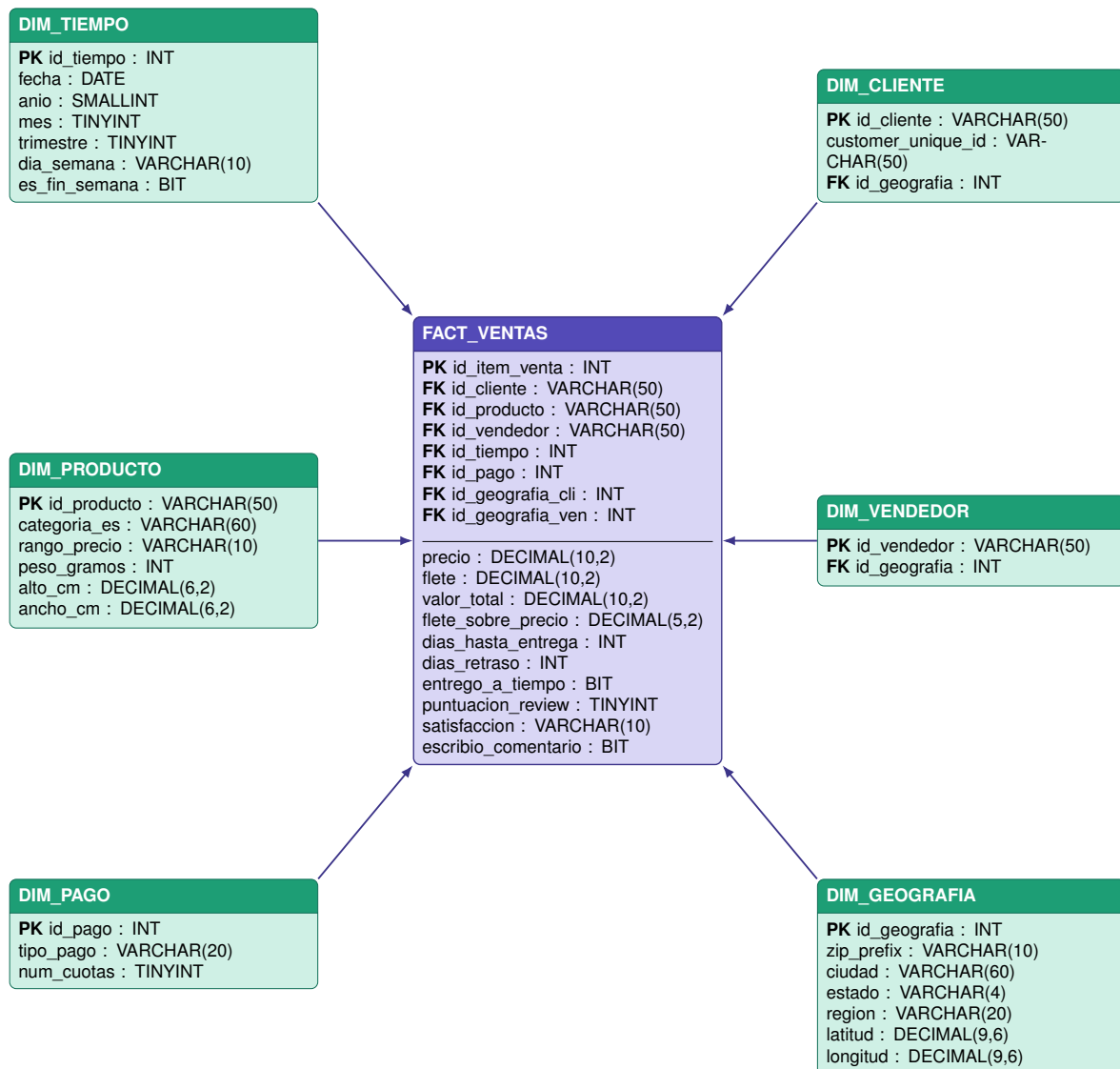


Figura 3: Modelo físico: tablas, columnas, tipos de datos, claves primarias y foráneas.

4.2 Criterios de diseño físico

- **Tipos de datos ajustados al rango esperado.** Se usa TINYINT para valores de 0 a 255 (puntuación de reseña, número de cuotas, mes, trimestre), SMALLINT para el año, BIT para banderas booleanas (entrego_a_tiempo, escribio_comentario, es_fin_semana) y DECIMAL(10,2) para valores monetarios.
- **Índices en columnas de filtrado frecuente.** Se definen índices secundarios sobre id_tiempo, id_producto y id_vendedor en la tabla de hechos, que son los tres filtros más usados por el dashboard del Problema 1, 3 y 4 respectivamente.
- **Doble conexión a la dimensión geografía.** La tabla de hechos referencia DIM_GEOGRAFIA dos veces: una por el lado del cliente (id_geografia_cli) y otra por el lado del vendedor (id_geografia_ven). Esta técnica de role-playing es la que permite construir las rutas origen→destino del análisis logístico.

4.3 Definición en SQL (DDL)

A continuación se presenta el script completo de creación de las seis tablas del modelo. Está diseñado para SQL Server, pero es fácilmente portable a PostgreSQL o MySQL con ajustes menores de sintaxis.

Listing 1: Creación de las tablas de dimensiones.

```
CREATE TABLE DIM_TIEMPO (
    id_tiempo          INT          NOT NULL PRIMARY KEY,
    fecha              DATE          NOT NULL,
    anio               SMALLINT     NOT NULL,
    mes                TINYINT      NOT NULL,
    trimestre          TINYINT      NOT NULL,
    dia_semana         VARCHAR(10)  NOT NULL,
    es_fin_semana      BIT          NOT NULL
);

CREATE TABLE DIM_GEOGRAFIA (
    id_geografia       INT          IDENTITY(1,1) PRIMARY KEY,
    zip_prefix         VARCHAR(10)  NOT NULL,
    ciudad             VARCHAR(60),
    estado             VARCHAR(4)   NOT NULL,
    region             VARCHAR(20)  NOT NULL,
    latitud            DECIMAL(9,6),
    longitud           DECIMAL(9,6)
);

CREATE TABLE DIM_CLIENTE (
    id_cliente         VARCHAR(50)  NOT NULL PRIMARY KEY,
    customer_unique_id VARCHAR(50)  NOT NULL,
    id_geografia       INT          NOT NULL,
    FOREIGN KEY (id_geografia) REFERENCES DIM_GEOGRAFIA(id_geografia)
);

CREATE TABLE DIM_VENDEDOR (
    id_vendedor        VARCHAR(50)  NOT NULL PRIMARY KEY,
    id_geografia       INT          NOT NULL,
    FOREIGN KEY (id_geografia) REFERENCES DIM_GEOGRAFIA(id_geografia)
);

CREATE TABLE DIM_PRODUCTO (
    id_producto        VARCHAR(50)  NOT NULL PRIMARY KEY,
    categoria_es       VARCHAR(60),
    rango_precio       VARCHAR(10),
    peso_gramos        INT,
    alto_cm            DECIMAL(6,2),
    ancho_cm           DECIMAL(6,2)
);

CREATE TABLE DIM_PAGO (
    id_pago            INT          IDENTITY(1,1) PRIMARY KEY,
    tipo_pago          VARCHAR(20)  NOT NULL,
    num_cuotas         TINYINT      NOT NULL
);
```

Listing 2: Creación de la tabla de hechos y sus índices.

```
CREATE TABLE FACT_VENTAS (
    id_item_venta      INT          IDENTITY(1,1) PRIMARY KEY,
    id_cliente         VARCHAR(50)  NOT NULL,
    id_producto        VARCHAR(50)  NOT NULL,
    id_vendedor        VARCHAR(50)  NOT NULL,
```



```

id_tiempo          INT          NOT NULL,
id_pago            INT          NOT NULL,
id_geografia_cli   INT          NOT NULL,
id_geografia_ven   INT          NOT NULL,
precio             DECIMAL(10,2) NOT NULL,
flete              DECIMAL(10,2) NOT NULL,
valor_total        DECIMAL(10,2) NOT NULL,
flete_sobre_precio DECIMAL(5,2),
dias_hasta_entrega INT,
dias_retraso       INT,
entrego_a_tiempo    BIT,
puntuacion_review   TINYINT,
satisfaccion        VARCHAR(10),
escribio_comentario BIT,
FOREIGN KEY (id_cliente) REFERENCES DIM_CLIENTE(id_cliente),
FOREIGN KEY (id_producto) REFERENCES DIM_PRODUCTO(id_producto),
FOREIGN KEY (id_vendedor) REFERENCES DIM_VENDEDOR(id_vendedor),
FOREIGN KEY (id_tiempo) REFERENCES DIM_TIEMPO(id_tiempo),
FOREIGN KEY (id_pago) REFERENCES DIM_PAGO(id_pago),
FOREIGN KEY (id_geografia_cli) REFERENCES DIM_GEOGRAFIA(id_geografia),
FOREIGN KEY (id_geografia_ven) REFERENCES DIM_GEOGRAFIA(id_geografia)
);

CREATE NONCLUSTERED INDEX idx_fact_tiempo
ON FACT_VENTAS(id_tiempo);
CREATE NONCLUSTERED INDEX idx_fact_producto
ON FACT_VENTAS(id_producto);
CREATE NONCLUSTERED INDEX idx_fact_vendedor
ON FACT_VENTAS(id_vendedor);

```

Próximos pasos

Con la arquitectura de datos definida, el equipo continuará con: (i) la exploración inicial de los nueve archivos CSV en Excel y la construcción de la hoja de diagnóstico de calidad; (ii) la implementación del proceso de ETL en Python con pandas y carga a SQL Server mediante sqlalchemy; y (iii) la validación de la tabla de hechos con consultas de prueba que respondan las preguntas centrales de cada uno de los cinco problemas de negocio, antes de avanzar hacia la construcción del dashboard ejecutivo.